DailyFlow: Design and Development of an AI-Integrated Android System for Intelligent Routine Management, Focus Enhancement, and Digital Distraction Prevention

Submitted by

MD. SALMAN AKAND
ID: 2110976160

MD. WASHIM AKRAM
ID: 2110276110

Course Code: CSE4102



Project proposal submitted in partial fulfillment of the requirements for the degree of

Bachelor of Science in
Computer Science and Engineering

Department of Computer Science and Engineering
University of Rajshahi

Date: November 9, 2026

ABSTRACT

DailyFlow is a native Android productivity application designed to combine routine planning, scheduled task management, application-level distraction control, short-form video interface interception, and local productivity analytics within a single mobile system. The core application is developed using Java and XML in Android Studio. SQLite, managed through SQLiteOpenHelper, stores structured task and daily productivity records, while SharedPreferences stores lightweight settings. AppBlockerService uses UsageStatsManager-based monitoring for selected applications, and ReelsBlockerService uses AccessibilityService events and exposed UI- node information to identify specified short-video interfaces in supported apps. The proposal also specifies a future AI-powered productivity assistant with seven proposed capabilities: daily routine suggestions, natural- language task creation, schedule optimisation, productivity insights, adaptive focus coaching, task-aware distraction prompts, and a conversational assistant. These AI capabilities are proposed extensions and are not claimed as part of the current core implementation. The proposed AI workflow uses limited application context, structured suggestions, deterministic validation, and user confirmation. Core operation remains local-first; any future external AI service would require explicit consent, minimal data sharing, secure credential handling, and a documented provider decision. The proposal presents requirements, architecture, data design, implementation methodology, testing, project planning, privacy safeguards, and expected outcomes. All empirical test outcomes remain pending until measured from the implemented build.

The core application uses Java and XML in Android Studio. SQLite, managed through SQLiteOpenHelper, stores structured task and daily productivity records. SharedPreferences stores lightweight settings such as profile name, theme, blocker preferences, selected blocked packages, and PIN-related configuration. The app-blocking and short-video detection services remain separate because they use different Android event sources and make different blocking decisions. The AI extension described in this proposal is optional and future-facing; the current build does not claim a deployed AI model, cloud API, or AI backend.

This proposal presents the system requirements, architecture, data model, algorithms, implementation methodology, testing strategy, project schedule, risks, and expected outcomes. Particular attention is given to Android permission requirements, foreground-service restrictions, changing third-party interface structures, accessibility privacy, and device/API compatibility. The core system is local-first and does not claim cloud synchronisation or machine-learning functionality. The proposed AI assistant is a separately identified extension whose provider and deployment mode remain undecided. Evaluation will measure core task-management correctness, blocking accuracy, detection latency, false-positive and false-negative rates, battery use, and usability; an additional AI test plan is specified for any later prototype. All test outcomes remain pending until empirical testing is completed.

TABLE OF CONTENTS

Section	Page
Preliminary Pages	2–4
Chapter 1: Introduction	4–6
Chapter 2: Literature Review and Related Work	7–8
Chapter 3: System Analysis and Requirements Engineering	9–10
Chapter 4: System Architecture and Design	11–21
Chapter 5: Implementation Methodology	22–23
Chapter 6: Testing and Evaluation	24–25
Chapter 7: Project Planning and Expected Outcomes	26–27
Chapter 8: Conclusion	28
References	28

LIST OF ABBREVIATIONS

Abbreviation	Meaning
API	Application Programming Interface
ERD	Entity Relationship Diagram
UI	User Interface
UX	User Experience
SDK	Software Development Kit
SQL	Structured Query Language
PIN	Personal Identification Number
DFD	Data Flow Diagram
FNR	False Negative Rate
FPR	False Positive Rate
OS	Operating System
CRUD	Create, Read, Update, Delete
SFV	Short-Form Video
FGS	Foreground Service
AI	Artificial Intelligence
LLM	Large Language Model
NLP	Natural Language Processing

CHAPTER 1: INTRODUCTION

1.1 Background of the Study

Smartphones support education, communication, work, entertainment, and personal organisation. At the same time, the same device can interrupt planned activities through notifications, habitual application switching, and continuously refreshed media feeds. Short-form video services present a particular interface challenge because users can move rapidly from one clip to another without leaving the host application. A person may therefore open a social application for a legitimate purpose, such as reading a message or checking a newsfeed, and then encounter a short-video surface. Research on digital self-control interventions has examined techniques including application blocking, goal setting, and usage visualisation. A systematic review identified a range of interventions while noting that their effectiveness requires careful evaluation [1]. Recent work on short-form video platforms has investigated how interface affordances and engagement relate to problematic social-media use [2]. These findings motivate tools that help users translate intentions into practical boundaries without assuming that every instance of short-video use is harmful.

1.2 Problem Statement

Routine planners commonly organise activities but do not necessarily intervene when a user moves into a distracting application or a short-video interface. General application blockers can restrict an entire application, which may be unnecessarily broad when the user needs legitimate functions such as messaging, ordinary posts, or newsfeeds. Conversely, usage-duration summaries provide awareness after or during use but may not interrupt a targeted interface at the moment it appears. The engineering problem is to design an Android application that combines schedule management with application-level and interface-level distraction controls. The solution must operate within Android permission and background-execution rules, preserve access to non-targeted application content where feasible, and remain transparent about the limitations of inspecting third-party UI structures.

1.3 Motivation

- Provide a single local application for planning tasks and supporting focus routines.
- Reduce the need to block an entire social application when the user's goal is to restrict a specific short-video surface.
- Keep the core application local-first and independent of AI services, while evaluating an optional AI assistant as a separately controlled future extension.
- Make daily completion and streak information visible so users can review their own routine consistency.
- Explore practical engineering limits of UsageStatsManager, AccessibilityService, and overlay-based controls across Android versions.

1.4 Proposed Solution

DailyFlow combines routine and task planning; smart focus and selected-application blocking; short-video interface detection; permission management; PIN-protected access; productivity analytics; and themes, alarms, and notifications. The core system uses Java, XML, Android SDK APIs, SQLite, and SharedPreferences. AppBlockerService monitors usage events and can display a blocking overlay for selected packages. ReelsBlockerService inspects accessibility events and exposed UI nodes for supported short-video indicators. This proposal additionally defines an optional future AI productivity assistant. It may help users prepare routines, convert natural-language instructions into task drafts, optimise schedules, interpret productivity records, adapt focus-session suggestions, provide task-aware distraction prompts, and ask questions through a chatbot. AI- generated changes must pass application validation and user confirmation before any task or schedule change is saved.

1.5 Aim and Objectives

1. Design a category-based task planner supporting start time, duration, recurrence, status, and task editing.
2. Implement time-overlap validation to prevent conflicting scheduled tasks.
3. Develop a foreground-service-based application blocker using UsageStatsManager and WindowManager overlays.
4. Implement accessibility-based detection rules for selected YouTube Shorts, Facebook Reels, Instagram Reels, and TikTok interfaces.
5. Preserve supported non-targeted interactions through application-specific checks and comment-section exceptions.
6. Store structured task and daily summary records in SQLite and lightweight preferences in SharedPreferences.
7. Provide PIN-based access control, theme preferences, reminders, notifications, and productivity summaries.
8. Evaluate functional correctness, detection quality, response time, compatibility, resource consumption, and usability.
9. Specify a future AI assistant module covering routine planning, natural-language task creation, schedule optimisation, productivity analysis, adaptive focus coaching, distraction-prevention prompts, and conversational task support, with validation, privacy, and user-confirmation controls.

1.6 Scope and 1.7 Significance

The core implementation is limited to Android devices and the stated Java/XML application, local SQLite storage, SharedPreferences, and Android system APIs. The core build does not include cloud synchronisation, remote administration, or an operational AI model. The proposal adds a separately identified future AI assistant design; its model/provider, deployment mode, and any backend are not yet selected and are subject to feasibility, privacy review, and project approval. The seven AI capabilities are therefore proposed scope rather than completed functionality. If implemented later, the AI module will receive only the minimum task, preference, or aggregate-statistics context required for a request. It will not receive raw accessibility trees, private messages, or unrelated screen content. AI-generated changes will require user confirmation and will be checked by the existing deterministic scheduling rules. Supported short-video rules remain limited to known UI identifiers and observable node patterns; they do not guarantee detection across every release, device, language, accessibility

configuration, or application variant. The intended compatibility range is Android 10?15, subject to API-level testing. The project remains significant as a software-engineering integration exercise involving local data management, Android services, special permissions, accessibility inspection, overlays, alarms, analytics, and a clearly bounded future AI design.

CHAPTER 2: LITERATURE REVIEW AND RELATED WORK

2.1 Productivity and Routine Management Applications

Routine-management applications typically support task entry, scheduling, reminders, recurrence, categorisation, and completion tracking. Their primary data model is a collection of user-defined activities with time and status attributes. Such applications can help users externalise intentions and make progress visible. However, a task list alone does not necessarily control what happens when a user switches to another application during a scheduled activity.

2.2 Digital Well-being and App Blocking Solutions

Digital self-control systems use several intervention types, including usage dashboards, time limits, application or website blocking, goal setting, and friction before access. Lyngs et al. describe a range of digital self-control interventions and highlight the need to understand how different designs support users in managing distracting media multitasking [1]. General app blockers are direct because they restrict access to a package or service, but they may also prevent legitimate use of that application.

2.3 Short-Form Video and Digital Distraction

Short-form video services commonly present vertically navigable streams, recommendation-driven content, and low-friction transitions between clips. Roberts and David studied technological affordances, engagement, and social-media addiction in relation to TikTok, Instagram Reels, and YouTube Shorts using a survey of college students [2]. Their survey design does not establish that the platforms cause addiction. This proposal therefore uses the more neutral terms 'digital distraction' and 'potentially unwanted short-video engagement' rather than diagnosing addiction. A 2026 systematic review of short-form video, cognitive and mental-health outcomes reported preliminary associations in the included literature while noting heterogeneity and methodological limitations, including cross-sectional designs and self- report measures [3]. This supports further investigation but does not justify a deterministic claim that all short-video exposure damages attention or mental health.

2.4 Limitations of Existing Approaches

- Whole-application blocking may be too broad when a user needs messaging, ordinary posts, or newsfeeds within the same application.
- Usage dashboards can inform users about behaviour but may not intervene at the exact interface that the user wishes to avoid.
- Application interfaces change over time, making fixed resource identifiers and node-text rules maintenance-sensitive.
- Android restricts sensitive capabilities through explicit user permissions and background-execution rules.
- Intervention effectiveness varies by design and user context; technical interception alone does not establish improved academic performance or wellbeing.

2.5 Research Gap

The project addresses an implementation-oriented gap: integrating routine scheduling with two levels of Android distraction control. One level targets selected application packages through usage-event monitoring and overlays. The other targets selected short-video interfaces within supported applications through accessibility-node inspection. The system also retains ordinary planner functions and local productivity summaries. The project does not claim that no comparable application exists; rather, it evaluates a specific combination of capabilities and the practical constraints of implementing them on Android.

2.6 Comparative Analysis

Table 2.1. Comparative analysis of conventional planners, general app blockers, and DailyFlow.

Capability	Conventional planner	General app blocker / digital wellbeing	DailyFlow
Task categories and schedule	Common	Usually limited or absent	Included
Task recurrence and status	Common	Not primary function	Included
Whole-app restriction	Usually absent	Common	Selected package blocking
Short-video interface detection	Usually absent	Varies; often not app-specific	Accessibility rules for named interfaces
Preserve ordinary feed/comment use	Not applicable	May block whole app	Exception logic where detectable
Local productivity summaries	Varies	Usage-focused summaries common	Daily and weekly task analytics
Platform dependency	App-specific	Android / OS-specific	Android 10–15, subject to testing

CHAPTER 3: SYSTEM ANALYSIS AND REQUIREMENTS ENGINEERING

3.1 Existing System Analysis

The existing user workflow may involve separate tools for task planning, reminders, and limiting social-media usage. This separation can require users to maintain schedules in one place and configure restrictions elsewhere. Whole-app restrictions can also conflict with legitimate use of the same application. DailyFlow proposes an integrated local workflow, while recognising that interface-specific interception is constrained by the information third-party applications expose to Android accessibility services.

3.2 Proposed System Analysis

The user creates tasks with categories, start times, durations, recurrence, and focus flags. The planner validates time intervals before saving. DatabaseHelper persists task records in SQLite, while PreferenceManager manages lightweight settings. AppBlockerService processes UsageStatsManager events and compares observed package names with the selected blocked-package list. ReelsBlockerService processes accessibility events and inspects exposed nodes for application-specific identifiers and text. AlarmHelper schedules reminders, and NotificationManager presents alerts and the daily summary. A proposed future AI assistant may read a limited, permission-controlled context from these existing modules and return structured suggestions; it will not replace the current task manager, database, blocker services, or deterministic validation logic.

3.3 Functional Requirements

Table 3.1. Functional requirements.

ID	Functional requirement
FR-01	Create, edit, view, and delete tasks.
FR-02	Assign one of the specified categories to a task.
FR-03	Configure start time, duration, and Daily/Weekly/None recurrence.
FR-04	Reject overlapping scheduled intervals using the defined interval test.
FR-05	Manage Pending, Ongoing, Completed, Skipped, and Locked task states.
FR-06	Select applications to block and enable or disable the app blocker.
FR-07	Monitor available usage events and display an overlay for a selected blocked package.
FR-08	Inspect accessibility events and apply supported app-specific short-video detection rules.
FR-09	Apply comment-section and other defined exceptions before an automatic Back action.
FR-10	Validate Usage Access and overlay permissions before opening the main interface.
FR-11	Provide four-digit PIN-based access protection.
FR-12	Calculate daily completion percentages, weekly summaries, and consecutive completion streaks.
FR-13	Allow the user to select a dark or light theme.
FR-14	Support task reminders and a 10:00 PM daily summary notification.

3.4 Non-Functional Requirements

Table 3.2. Non-functional requirements and evaluation criteria.

Attribute	Requirement / evaluation criterion
Reliability	Task records and settings should remain consistent after normal restarts; services should handle unavailable permissions gracefully.
Performance	Measure event-to-decision and decision-to-action latency; ~500 ms is a target for Back response, not a guarantee.
Compatibility	Evaluate Android 10–15 with API-specific permission and service requirements.
Usability	Permission explanations, blocker states, task conflicts, and PIN errors should be understandable.
Privacy	Disclose usage and accessibility access; limit information to local functions.
Security	Protect PIN verification data; plaintext PIN storage is not recommended.
Maintainability	Keep app-specific detection rules isolated and documented for updates.
Resource efficiency	Measure CPU, memory, service wakeups, and battery impact.

3.5 Hardware and Software Requirements

Table 3.3. Hardware and software requirements.

Category	Requirement / technology
Device	Android smartphone running Android 10–15 for target evaluation.
Memory / storage	At least 2 GB RAM recommended for practical testing; sufficient local storage for app and SQLite data.
Development computer	Computer capable of running Android Studio and Android SDK; USB or wireless debugging.
IDE / language	Android Studio; Java.
UI / platform	XML layouts and Android SDK; target compatibility evaluated across API levels 29–35.
Structured storage	SQLite using SQLiteOpenHelper / DatabaseHelper.java.
Preferences	SharedPreferences using PreferenceManager.java.
Monitoring / inspection	UsageStatsManager / UsageEvents; AccessibilityService API.
Blocking / background	WindowManager system overlay; Android Foreground Service.
Scheduling / testing	AlarmManager, NotificationManager, emulator, physical devices, Logcat and manual/automated tests.

3.6 Feasibility Analysis

Technical feasibility

The required Android APIs exist, but their use is permission-dependent and version-sensitive. UsageStatsManager requires user-granted Usage Access. Drawing over other apps requires special overlay permission. AccessibilityService must be explicitly enabled and configured for relevant events and node information. Foreground-service start restrictions and service- type requirements vary with target SDK. Android 15 narrows an overlay-related exemption for starting foreground services from the background [4]?[6].

Operational and economic feasibility

The application can be operated by users who understand and grant the requested special permissions. Onboarding must explain why each capability is required and provide a way to disable blockers. Accessibility access can expose on-screen information, so unrelated content must not be collected or transmitted. The design uses the Android development environment and local storage; no paid cloud infrastructure is required. Development cost is primarily student time and access to suitable Android test devices.

Schedule feasibility

3.7 Use Case Descriptions

Table 3.4. Use case descriptions.

Use case	Actor	Precondition	Outcome
Manage routine task	User	Application opened and access unlocked	Task created, updated, completed, skipped or deleted.
Configure blocker	User	Required permissions granted	Selected package list and blocker preference saved.
Intercept short-video interface	System / User	Accessibility service enabled; supported app active	Target interface identified; Back action may be invoked.
Review productivity	User	Task and history data exist	Daily/weekly completion and streak displayed.
Configure reminder	User	Task has a valid schedule	Alarm scheduled and local notification can be delivered.
Unlock protected area	User	PIN protection configured	PIN validated; protected screen opens if valid.

The primary actor is the device owner. Android system settings act as the external authority for special-permission grants. Other applications are monitored targets, not trusted system components. UsageStatsManager and AccessibilityService provide system-mediated observations; NotificationManager and WindowManager provide system-mediated output surfaces.

CHAPTER 4: SYSTEM ARCHITECTURE AND DESIGN

4.1 Overall System Architecture

DailyFlow follows a local-first, component-oriented Android architecture. MainActivity coordinates the application interface and permission gate. DatabaseHelper owns SQLite CRUD operations. PreferenceManager owns lightweight preference access. AppBlockerService and ReelsBlockerService operate as separate monitoring paths because package-level foreground-use detection and in-app interface inspection solve different problems. AlarmHelper coordinates task reminder scheduling. Android system APIs provide usage events, accessibility events, overlay windows, alarms, and notifications.

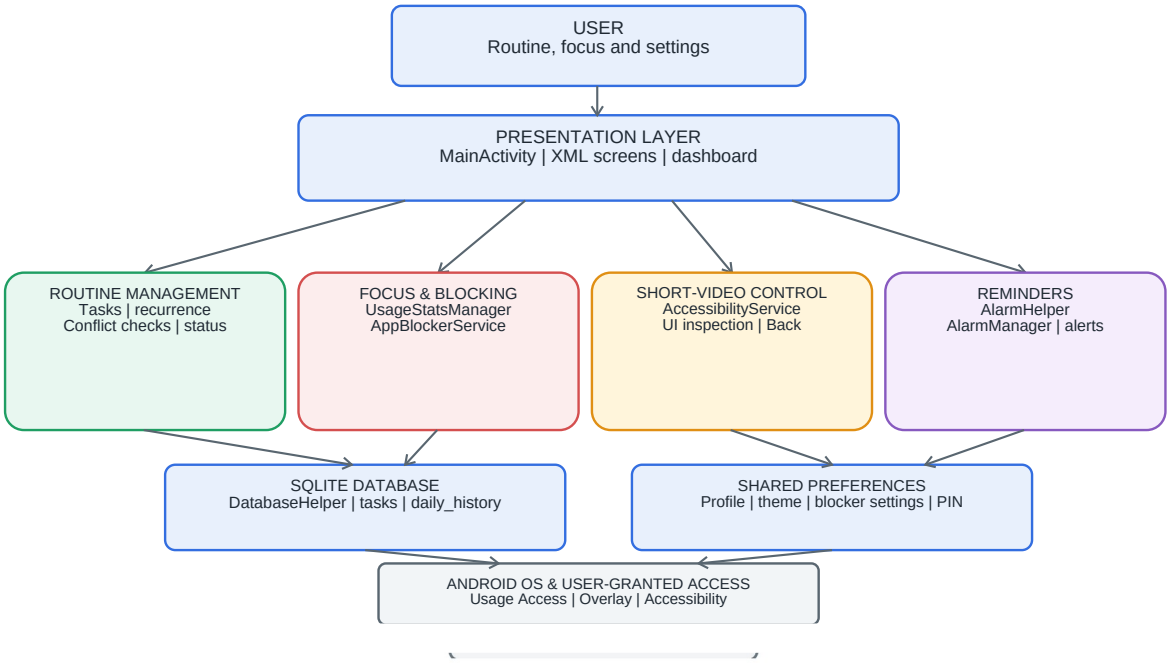


Figure 4.1. Overall system architecture and principal data/control paths.

MainActivity is the user-facing coordinator, while persistence, monitoring, and scheduling are delegated to dedicated components. AppBlockerService identifies a foreground package; ReelsBlockerService examines the visible interface within a target application. The services use local preferences for configuration. No server or cloud component is present.

4.2 Architectural Principles

- Separation of concerns: UI, data access, background monitoring, and platform APIs have distinct responsibilities.
- Local-first operation: task records and configuration remain on the device in SQLite and SharedPreferences.
- Independent monitoring paths: whole-app blocking and short-video interface detection are separate functions.
- Permission transparency: sensitive Android capabilities are requested through explicit user-controlled settings.
- Bounded detection claims: supported identifiers are version-sensitive and must be maintained and tested.

4.3 Layered Architecture

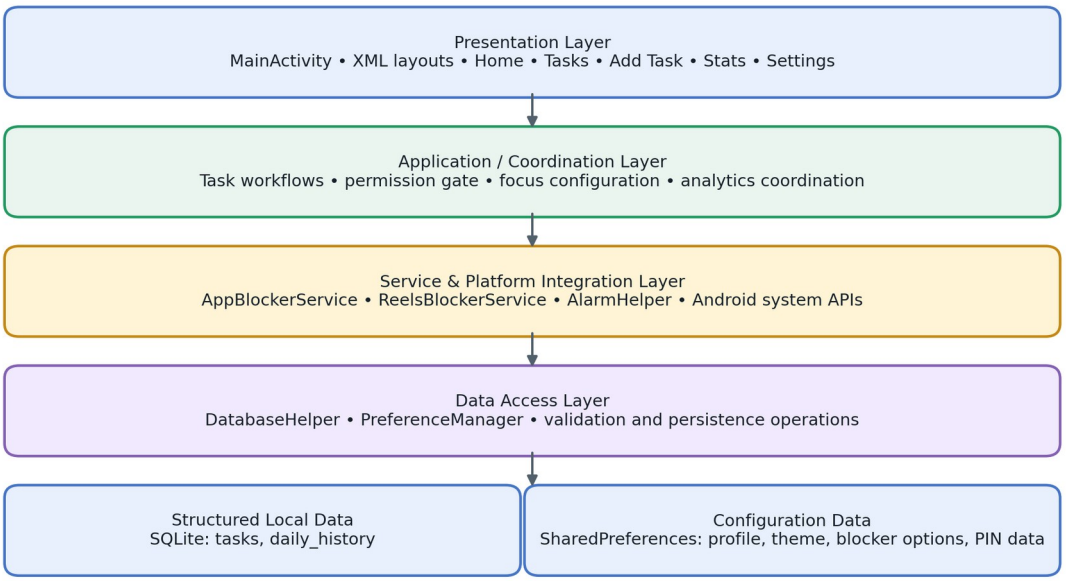


Figure 4.2. Logical layered architecture of DailyFlow.

4.3.1 Layer Descriptions

- Presentation: MainActivity and XML screens display Home, Tasks, Add Task, Stats, and Settings.
- Application/coordination: validates task input, coordinates permission gating, applies scheduling rules, and requests analytics.
- Service/platform integration: AppBlockerService, ReelsBlockerService, AlarmHelper, and Android APIs.
- Data access: DatabaseHelper and PreferenceManager provide controlled persistence access.
- Persistence: SQLite stores relational task/history records; SharedPreferences stores small configuration values.

4.4 Android Component Responsibilities

Table 4.1. Android application component responsibilities.

Component	Responsibility	Inputs	Outputs / owned data
MainActivity	Permission gate, navigation, user interaction	User actions and permission state	UI state; calls managers and services
DatabaseHelper	SQLite schema and CRUD operations	Task and daily summary values	tasks and daily_history rows
PreferenceManager	Read/write settings	Profile, theme, blocker switches, package list, PIN configuration	SharedPreferences values
AppBlockerService	Monitor usage events and block selected packages	UsageEvents and preferences	Overlay request and service notification
ReelsBlockerService	Inspect accessibility events and detect target interfaces	AccessibilityEvent, UI node tree and rules	Detection decision and Back action
AlarmHelper	Schedule/cancel task reminders	Task ID and scheduled time	AlarmManager PendingIntent configuration
UsageStatsManager	Expose usage-event history	Time interval query	UsageEvents
WindowManager	Attach/remove overlay window	Overlay view and parameters	Visible blocking surface
AccessibilityService	Receive configured UI events and expose node tree	Third-party app UI events	AccessibilityEvent and node information

4.5 Database Architecture and Data Ownership

SQLite is used for structured, queryable records. The tasks table stores individual scheduled activities. The daily_history table stores date-level aggregate counts. SharedPreferences is not a substitute for the relational database: it is suitable for small independent settings, while SQLite supports multiple records, typed columns, filtering, sorting, and transactional updates. No foreign key is specified between daily_history and tasks because the supplied design does not define one.

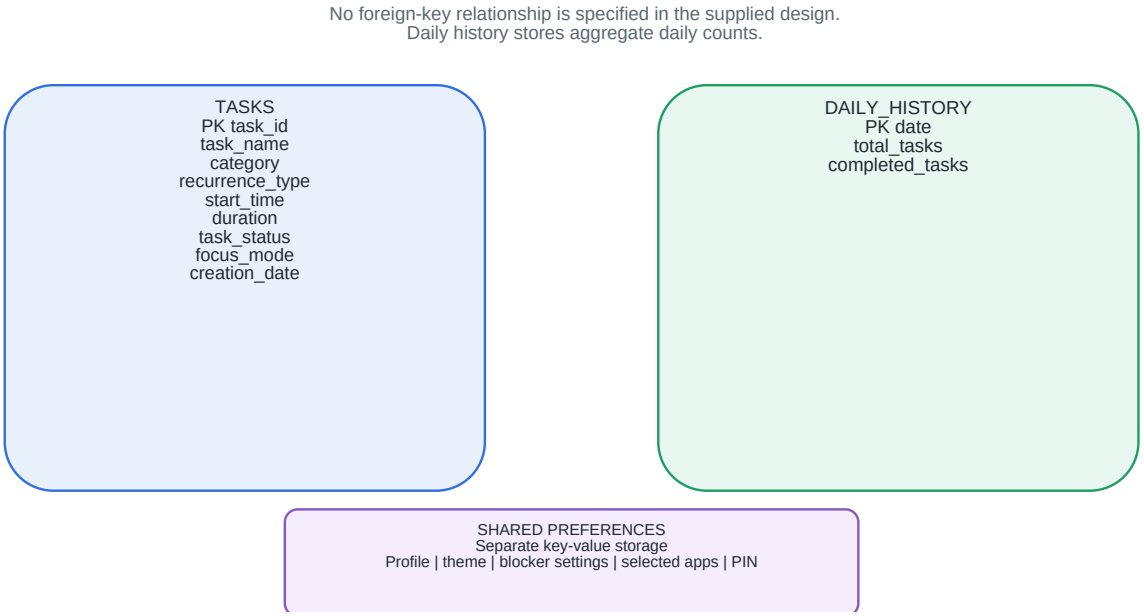


Figure 4.3. SQLite entity relationship model; SharedPreferences is separate key-value storage.

4.5.1 Proposed Logical Database Schema

Table 4.2. Proposed logical SQLite database schema.

Table	Field	Type	Constraint / meaning
tasks	task_id	INTEGER	Primary key; unique task identifier.
tasks	task_name	TEXT	Required task title.
tasks	category	TEXT	Study, Religious Activities, Exercise, Work, Meals, Sleep, Others.
tasks	recurrence_type	TEXT	Daily, Weekly, or None.
tasks	start_time	TEXT	Required local time representation.
tasks	duration	INTEGER	Positive duration in minutes.
tasks	task_status	TEXT	Pending, Ongoing, Completed, Skipped, or Locked.
tasks	focus_mode	INTEGER	Boolean-like 0/1 flag.
tasks	creation_date	TEXT	Date/time record created.
daily_history	date	TEXT	Primary key proposed for one aggregate row per date.
daily_history	total_tasks	INTEGER	Non-negative task count.
daily_history	completed_tasks	INTEGER	Non-negative and not greater than total_tasks.

The schema formalises only fields supplied in the project specification. Exact column names, SQL types, defaults, indexes, and constraints must be aligned with DatabaseHelper.java before final submission. The ER diagram intentionally has no relationship line between the two entities.

4.6 Data Flow Diagrams

4.6.1 Context-Level DFD (Level 0)

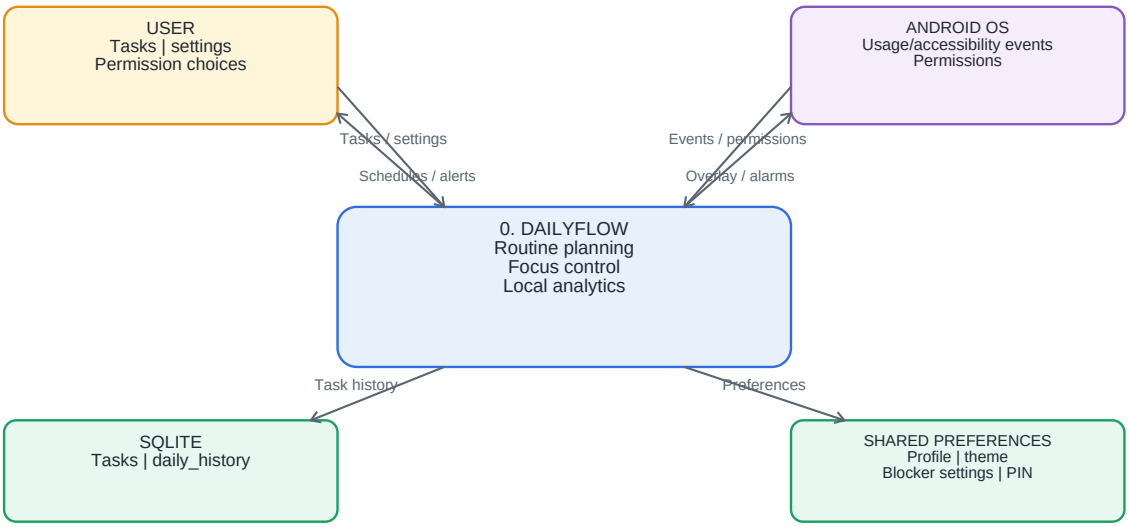


Figure 4.4. Context-level data flow diagram.

At context level, DailyFlow is treated as one process. The user provides tasks, settings, and permission choices and receives schedules, status, alerts, and analytics. Android supplies usage/accessibility events and permission state. SQLite and SharedPreferences are represented as separate local stores.

4.6.2 Level 1 DFD

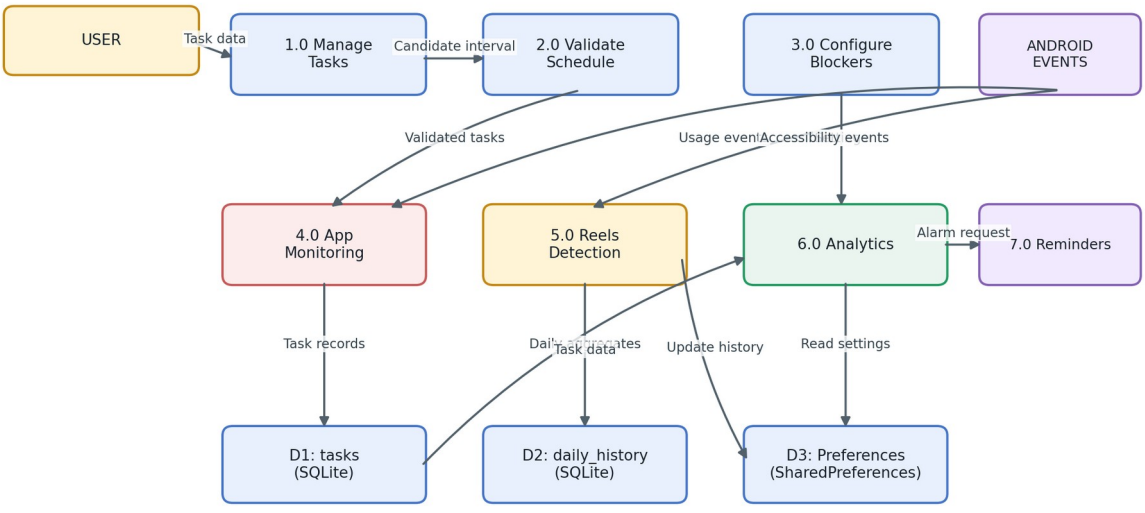


Figure 4.5. Level 1 data flow diagram showing major processes and data stores.

4.7 Use Case Diagram and Actors

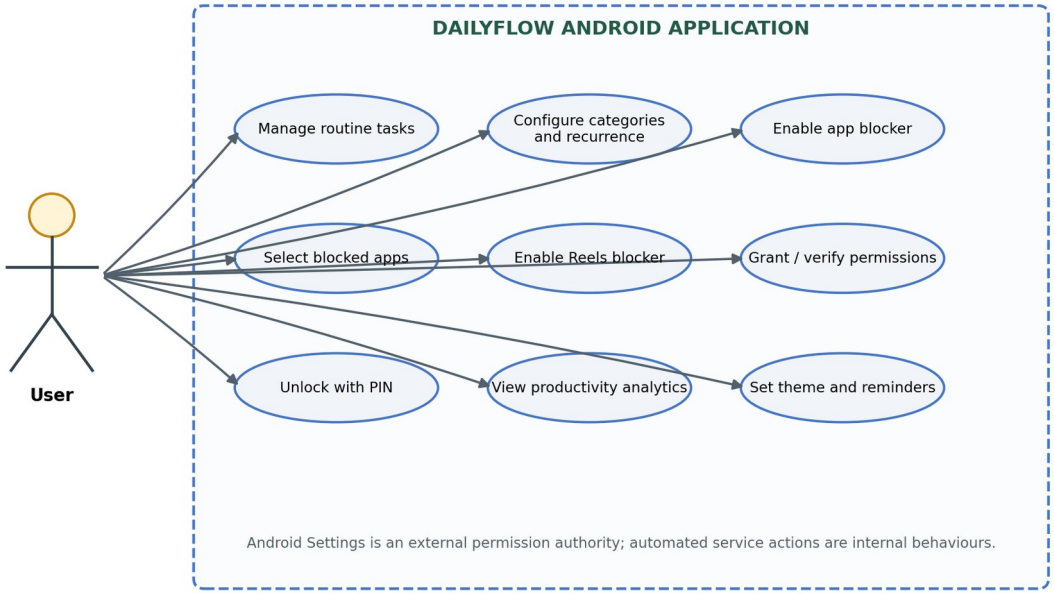


Figure 4.6. DailyFlow use case diagram.

The user is the principal actor and interacts with task management, categories and recurrence, application blocking, blocked- app selection, Reels blocking, permission verification, PIN access, productivity analytics, themes, and reminders. Android Settings is an external permission authority, but the user initiates the permission workflow from DailyFlow. Automated service actions are internal system behaviours rather than separate human actors.

4.8 Module-Based Architecture

Table 4.3. Module-based architecture and responsibilities.

Module	Responsibility
Task Planner	Task creation, category assignment, recurrence, duration, conflict checks, and status transitions.
App Blocking	Selected package list, UsageStatsManager event processing, and overlay lifecycle.
Reels Detection	Application-specific detection rules, UI-node inspection, comment-section exception, and Back action.
Permission Management	Validation of Usage Access and overlay permissions; separate accessibility permission check.
Security	Four-digit PIN validation and protected access.
Analytics	Daily/weekly summaries, completion percentage, and streak derivation.
Notifications	Task reminders and daily 10:00 PM summary.

4.9 Class Diagram

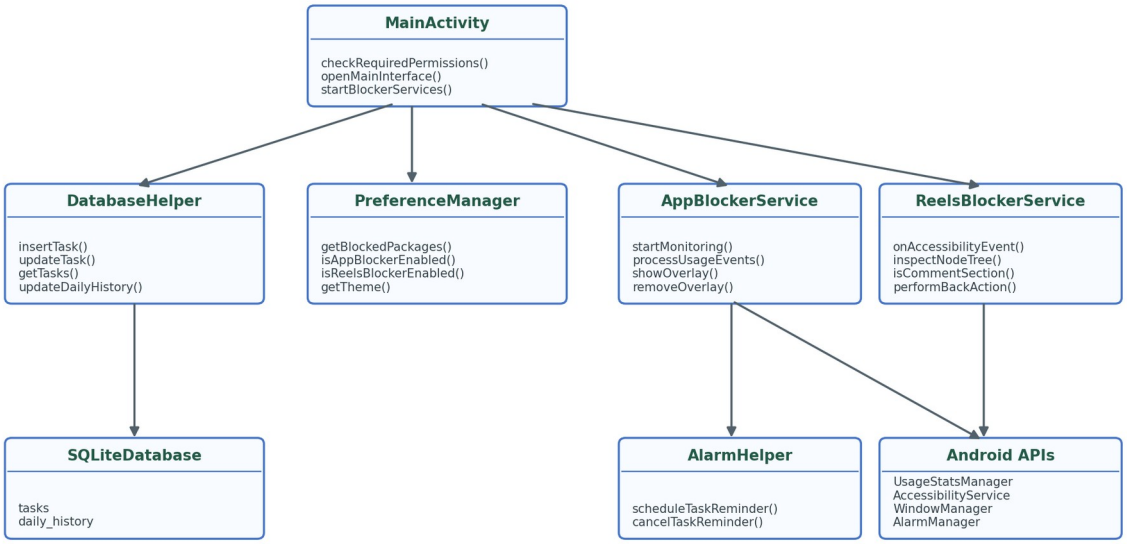


Figure 4.7. Conceptual class diagram of the principal application classes and dependencies.

The diagram identifies responsibilities and dependencies rather than asserting exact method signatures. MainActivity coordinates the interface and delegates structured persistence, preference access, blocking, accessibility inspection, and reminder scheduling to dedicated components. The actual source code should be used to refine method names, visibility, constructors, and class relationships before the final defence.

4.10 App-Blocking Sequence and Workflow

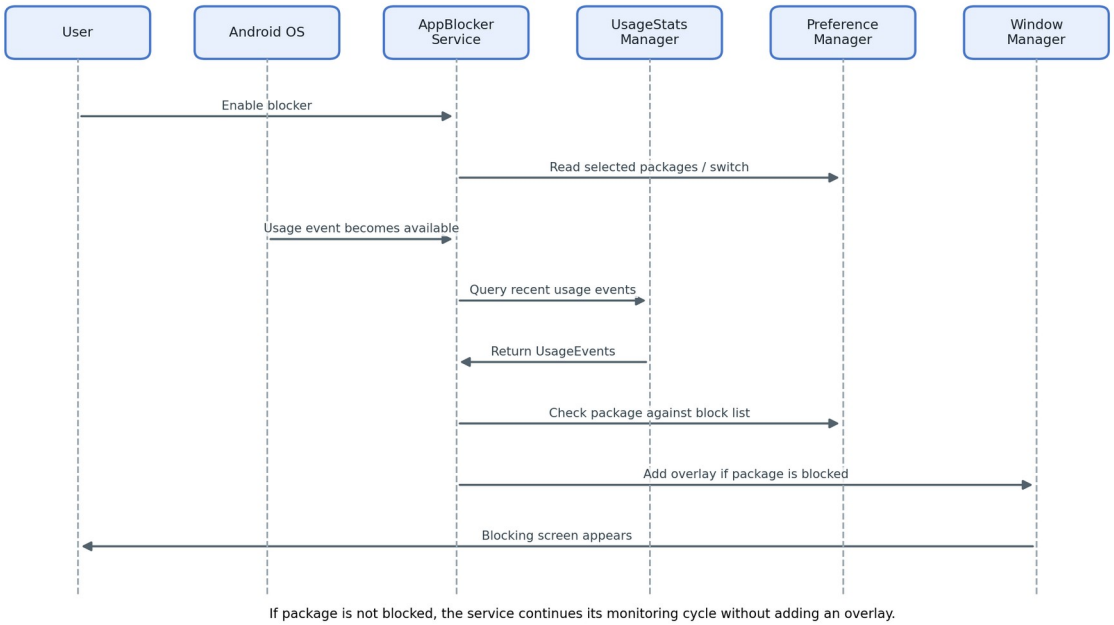


Figure 4.8. App-blocking sequence diagram.

AppBlockerService runs as a foreground service with an ongoing notification. It queries UsageStatsManager for recent UsageEvents and determines the most recent relevant foreground package from the available event sequence. If the package matches the selected block list and the blocker is enabled, it asks WindowManager to attach a TYPE_APPLICATION_OVERLAY view. The overlay is removed when the restricted package is no longer the active target or when the blocker is disabled. UsageStatsManager is event-based and does not guarantee an instantaneous foreground-app callback. Polling interval, event availability, device scheduling, and service lifecycle affect detection latency. The implementation must avoid duplicate overlay windows and handle permission revocation and service destruction safely.

4.10.1 Application-Level Blocking Flow

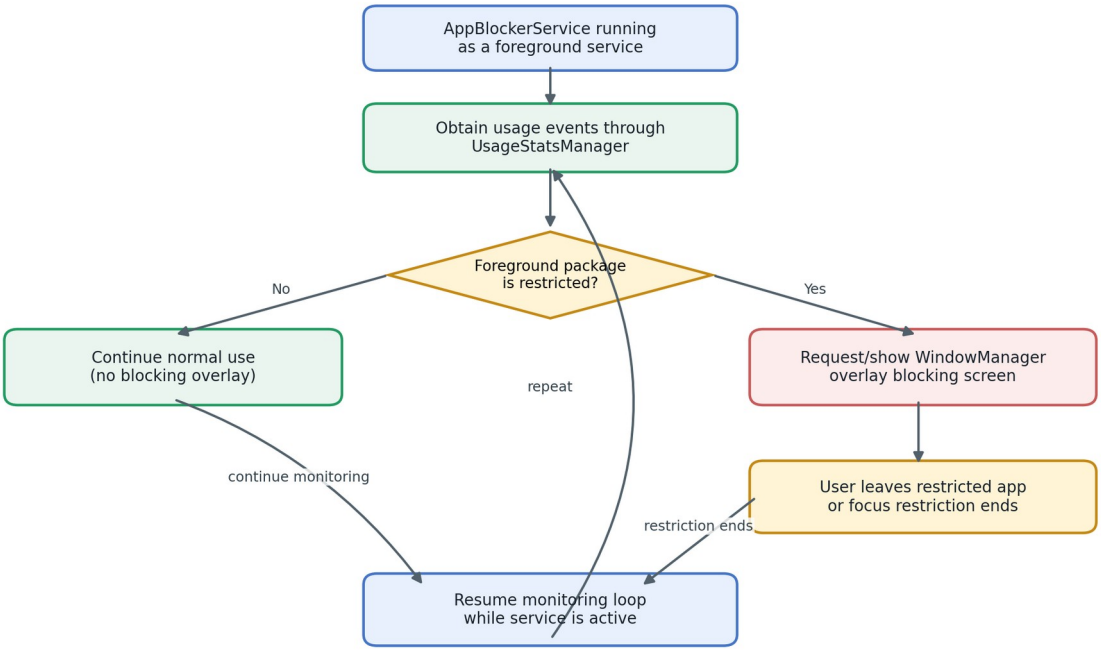


Figure 4.9. Application-level blocking workflow.

4.11 Real-Time Reels Detection Architecture

ReelsBlockerService receives configured AccessibilityEvents, identifies the active package, and inspects available accessibility nodes. It evaluates application-specific identifiers and text markers. The comment-section exception is checked before an automatic Back action to reduce false positives and preserve ordinary interactions where detectable.

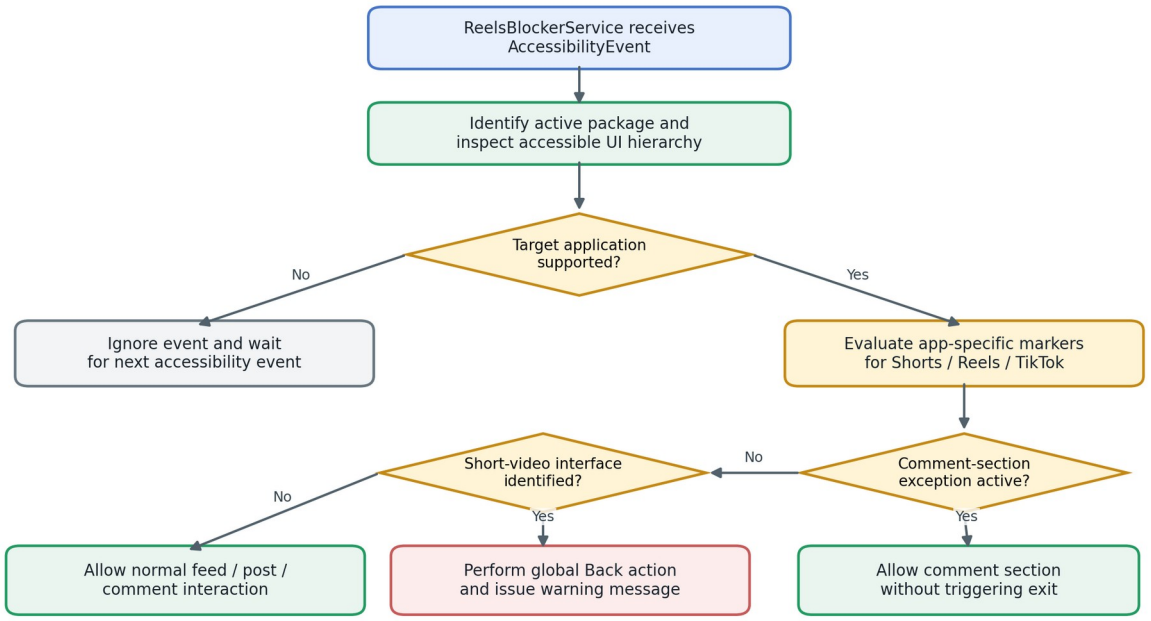


Figure 4.10. Accessibility-based short-video detection and exception handling.

4.10.1 Application-Specific Detection Indicators

Table 4.4. Application-specific short-video detection indicators.

Target application	Indicators described in the project
YouTube Shorts	reel_viewer; shorts_player; remix button; Create a Short; reel_recycler.
Facebook Reels	fb_shorts; reel_viewer; reels_video_view; reels_full_screen.
Instagram Reels	clips_viewer; clips_video_container; reels_viewer.
TikTok	Video-feed pager detection.

These identifiers are detection rules, not permanent platform interfaces. Third-party applications may rename, rearrange, or replace UI elements. Detection coverage must therefore be tested across app versions and maintained when structures change. Accessibility inspection is permission-dependent and should be limited to the stated distraction-prevention purpose. The approximately 500 ms Back-action response is a target to measure from event receipt to action invocation, not a guaranteed maximum latency. A zero-millisecond notification timeout cannot eliminate event-delivery, scheduling, node- inspection, or OS latency.

4.12 Task Scheduling and Conflict Detection

The task form collects a task name, category, recurrence type, start time, duration, and optional Focus Mode selection. The system calculates the end time and validates the input. It then checks the proposed interval against applicable existing task occurrences before inserting or updating the record through DatabaseHelper.

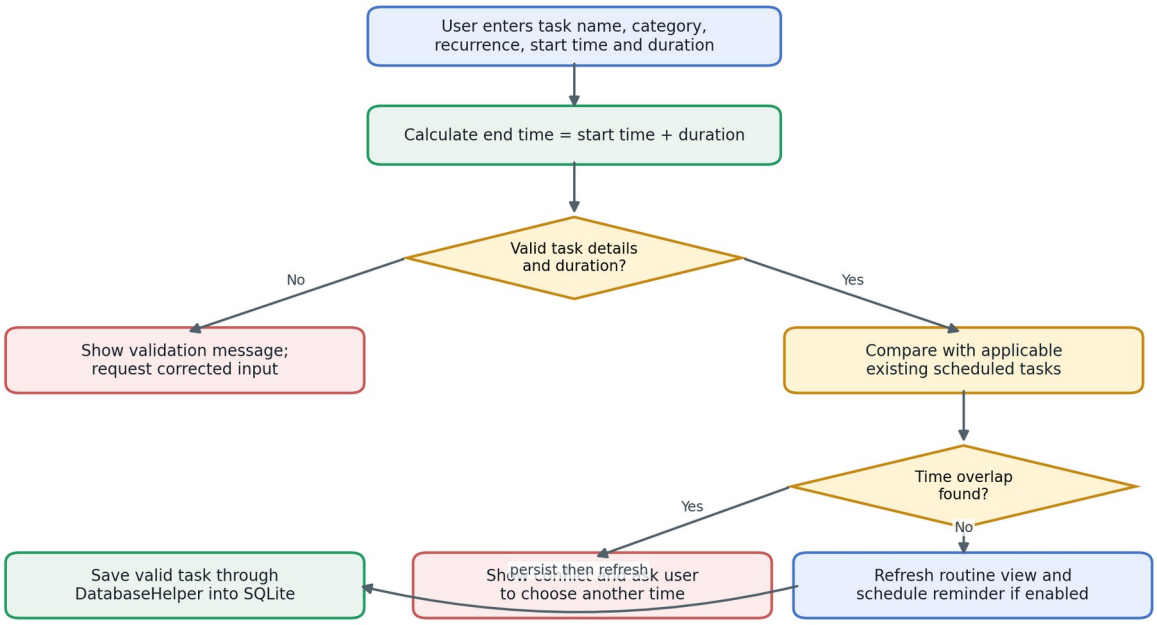


Figure 4.11. Task creation, validation and overlap-checking workflow.

4.11.1 Time-Overlap Detection

Each task is represented as a half-open interval A = [startA, endA), where endA = startA + duration. Two intervals overlap if and only if:

$$\mathbf{startA < endB \ \ AND \ \ startB < endA}$$

The strict inequalities mean that one task may end exactly when another begins without being considered an overlap. For example, 09:00?10:00 and 10:00?10:30 are adjacent but non-overlapping. However, 09:00?10:00 and 09:45?10:15 overlap. Recurring tasks must be compared against the relevant occurrence date after recurrence expansion.

4.13 Productivity Analytics Architecture

Daily completion percentage is completedTasks / totalTasks × 100. If totalTasks is zero, the application should avoid division by zero and display a defined neutral state such as 0% or 'No tasks scheduled'. Weekly analytics aggregate daily counts. A streak algorithm walks backward from the relevant date and counts consecutive days meeting the documented completion criterion. The project must define whether a qualifying day means at least one completed task, all scheduled tasks completed, or a threshold percentage.

4.14 Permission Management Architecture

On first launch, MainActivity checks whether Usage Access is granted and whether the app can draw overlays. If either is missing, the user is shown an explanation and directed to the relevant Android settings screen. On return, permission state is checked again; the main interface remains gated until both mandatory permissions are available. Accessibility permission is separately required for ReelsBlockerService and should be explained and checked before that feature is enabled.

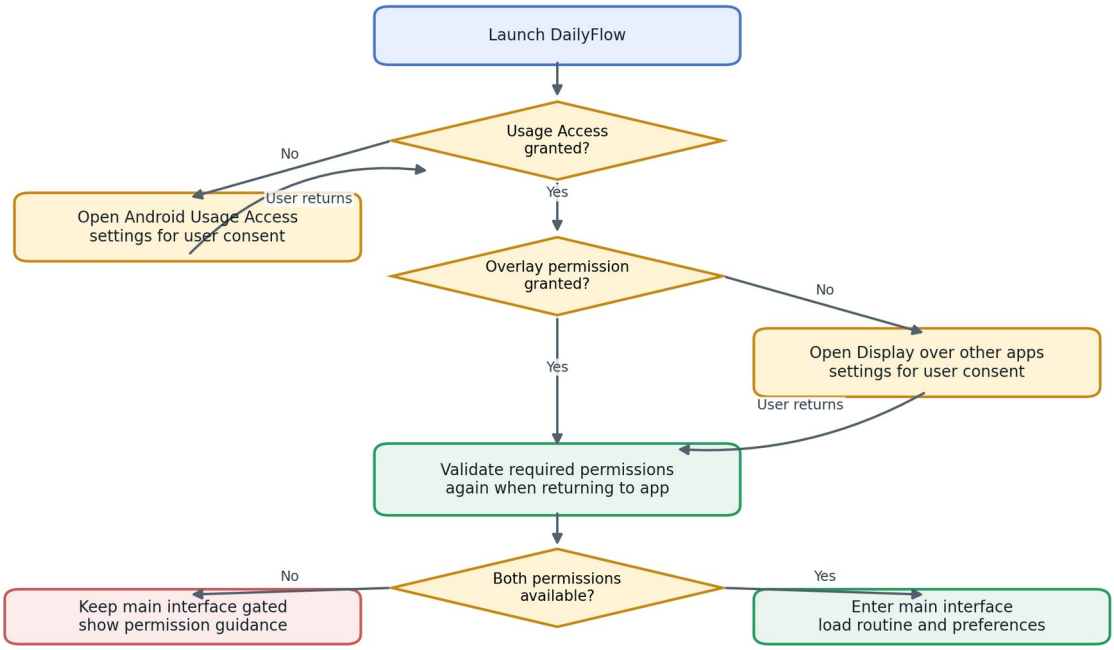


Figure 4.12. First-launch permission-gating workflow.

4.15 Android Permissions, Privacy and Compatibility

- Usage Access: explicit user consent is required; usage data can reveal app-use patterns and must be used only for the stated function.
- Display over other apps: special user-granted permission is required before showing a WindowManager overlay.
- Accessibility Service: explicit user enablement is required; UI access is privacy-sensitive and must not be used to collect unrelated content.
- Foreground service: comply with applicable notification, service-type, and background-start requirements across Android releases.
- Android 10–15: compatibility must be tested against API level, target SDK, device manufacturer, service startup, and overlay behaviour.

4.16 PIN Security

The supplied design stores four-digit PIN-related configuration through PreferenceManager in SharedPreferences. SharedPreferences is a configuration store, not a secure credential vault by itself. Plaintext PIN storage should be avoided. A recommended improvement is to store a salted, iteratively derived verifier (for example, PBKDF2 with a per-installation random salt), rate-limit repeated attempts, and avoid logging PIN values. Android Keystore can protect cryptographic keys, but it does not automatically make a plaintext PIN safe. These are recommendations, not claims about already implemented behaviour.

4.17 Proposed AI-Powered Productivity Assistant

The AI assistant is designed as an optional application layer above the existing DailyFlow modules. It is not a replacement for MainActivity, DatabaseHelper, PreferenceManager, AppBlockerService, ReelsBlockerService, or AlarmHelper. Its purpose is to interpret a user's request and prepare a recommendation using a restricted context assembled from authorised application data. The exact model, provider, and deployment method remain open design decisions.

AI feature	Proposed use in DailyFlow	Control / limitation
AI Daily Routine Assistant	Suggests a balanced day plan from goals, preferences, available time, and existing commitments.	User reviews and confirms the draft.
AI Smart Task Creation	Converts plain-language instructions into structured task fields.	Missing/uncertain fields require clarification; normal task validation still applies.
AI Schedule Optimiser	Suggests conflict-free task order, alternative	Existing interval-overlap logic validates every

AI feature	Proposed use in DailyFlow	Control / limitation
AI Productivity Analyst	Explains completion rates, streaks, and category-level trends.	Uses recorded data only; reports insufficient history when appropriate.
AI Adaptive Focus Coach	Suggests focus duration and break intervals from permitted session outcomes.	Advice is optional and adjustable; no diagnosis or guaranteed performance claim.
AI Distraction Prevention Coach	Creates a short reminder related to the active task when a blocker event occurs.	Existing services detect/block; AI only personalises the message.
AI Personal Chatbot	Answers questions about today's tasks, pending work, tomorrow's routine, and progress.	Read-only answers use authorised data; task changes need confirmation.

Table 4.5. Proposed AI capabilities and their user-control requirements.

4.18 Proposed AI Integration Workflow

Figure 4.13 presents a controlled workflow. A request is first interpreted by a context builder, which selects only the minimum task, preference, or aggregated-history data needed. A future AI provider returns a structured suggestion. A validation layer checks required fields, allowed actions, and scheduling constraints. The user then previews and confirms the result. Only after confirmation does the existing application logic perform the database operation.

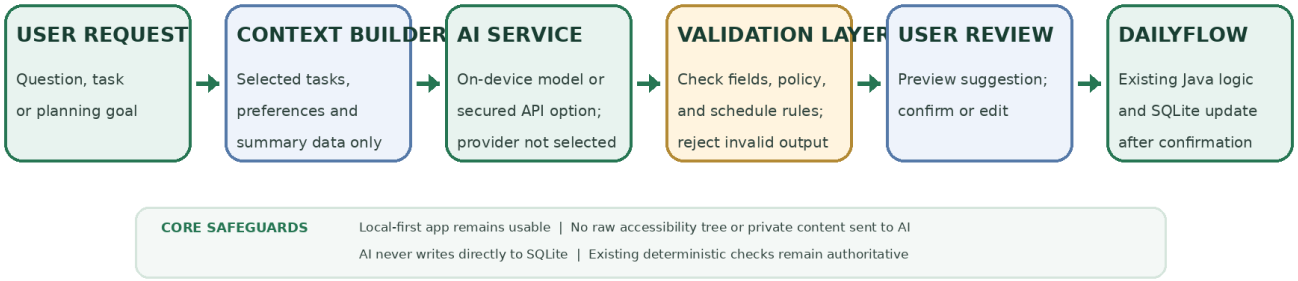


Figure 4.13. Proposed AI request, context, validation, and user-confirmation workflow.

4.19 AI Data Governance, Privacy, and Safety

The core app remains local-first. For a future AI implementation, an on-device model or a secured external AI service may be evaluated; no provider or deployment option is selected in this proposal. The design must apply the following safeguards:

- Data minimisation: provide only the task fields, user-selected preferences, or aggregated productivity values needed for the current request.
- No unrelated screen capture: raw AccessibilityNodeInfo trees, private messages, passwords, and unrelated third-party content must not be sent to an AI model.
- Explicit control: AI assistance must be opt-in, clearly labelled, and switchable off without disabling routine planning or blocking functions.
- Human approval: task creation, deletion, editing, and rescheduling must be previewed and confirmed by the user.
- Deterministic validation: existing conflict detection, permissions, task-state rules, and database constraints remain authoritative over generated output.
- Secure service integration: if a remote model is selected, requests must use an authenticated backend over TLS; provider secrets must not be embedded in the Android APK.
- Transparency and fallback: explain that AI output may be inaccurate; handle network/model failure gracefully and retain the existing manual workflow.

CHAPTER 5: IMPLEMENTATION METHODOLOGY

5.1 Development Methodology

An iterative and incremental development approach is appropriate because DailyFlow contains modules with different technical risks. The task planner and local database can be implemented and tested independently before integrating Android background services and accessibility-based detection. Each iteration should produce a testable increment, review defects, and update the design documentation.

5.2 System Development Life Cycle

Table 5.1. Development life-cycle stages and deliverables.

Stage	Main activities
1. Requirements and audit	Confirm source code, features, schema, target SDK, manifest declarations, and permissions.
2. Architecture and data design	Finalise component responsibilities, SQLite schema, SharedPreferences keys, and diagrams.
3. Core planner	Implement task CRUD, recurrence, conflict validation, and status management.
4. Analytics and reminders	Implement daily history, weekly summaries, streak logic, AlarmHelper, and notifications.
5. Distraction controls	Integrate permission gate, foreground app blocker, overlay, accessibility detection, and exceptions.
6. Integration and evaluation	Test end-to-end workflows, compatibility, latency, false positives/negatives, and battery impact.
7. Documentation and defence	Align proposal with source code, prepare evidence, final report, and demonstration.

5.3 Module Implementation Strategy

The planner module should be completed first because it establishes the task data model and scheduling rules. DatabaseHelper should centralise SQLite creation and CRUD operations. PreferenceManager should centralise configuration reads and writes. AppBlockerService and ReelsBlockerService should remain separate because they use different Android event sources and make different blocking decisions. AlarmHelper should schedule reminders from task data, while NotificationManager presents the user-facing alerts.

5.4 Database Implementation

SQLiteOpenHelper manages database creation and version upgrades. The tasks table stores individual activities; daily_history stores date-level aggregates. Updates to task status and daily aggregates should be kept consistent, preferably using transactions where multiple related writes are required. SharedPreferences is reserved for lightweight configuration rather than task lists or analytics history.

5.6 Android Service and API Implementation

5.6.1 Foreground Service and Usage Monitoring

AppBlockerService is intended to run as an Android foreground service with an ongoing notification. It queries UsageStatsManager for recent UsageEvents, interprets the available event sequence to infer the most recent foreground package, and compares the package name with the selected block list stored in SharedPreferences. UsageStatsManager provides usage-event history rather than a guaranteed instantaneous foreground callback; therefore, event polling and device scheduling influence response time.

5.6.2 Accessibility-Based Interface Inspection

ReelsBlockerService receives AccessibilityEvents and obtains accessible node information when available. It first checks the source package, then inspects resource identifiers, text, and relevant descendants. App-specific matching rules should be isolated so they can be updated when third-party applications change. Null nodes, duplicate events, incomplete trees, and non-target screens must be handled safely.

5.6.3 Overlay-Based Blocking

When AppBlockerService identifies a selected blocked package and the blocker is enabled, it requests WindowManager to attach a blocking overlay. The overlay should be removed when the restricted package is no longer the target or when the blocker is disabled. The implementation must check overlay permission, avoid duplicate window attachment, and handle permission revocation or service destruction.

5.6.4 Permission Handling

Usage Access and overlay access are special permissions controlled through Android settings rather than ordinary runtime permission dialogs. MainActivity must re-check permission state when resumed after the user returns from settings. Accessibility access is a separate requirement for Reels detection. A feature should not silently assume that its permission remains granted.

5.6.5 PIN, Alarm and Notification Management

PIN validation should compare the entered value with the locally stored verifier and protect sensitive screens. Plaintext storage is not recommended. AlarmHelper schedules a PendingIntent through AlarmManager; NotificationManager displays task reminders and the 10:00 PM daily summary. Exact-alarm access and notification permission requirements vary by Android version. On Android 14, SCHEDULE_EXACT_ALARM is denied by default for many newly installed applications targeting Android 13 or later; check canScheduleExactAlarms() and provide an appropriate fallback [8]. Android 13 and later also require runtime notification permission for ordinary notifications.

5.7 Development and Code Alignment

The final implementation record should confirm the actual service names, manifest declarations, notification channel, foreground-service type, UsageEvents event types and polling interval, detection predicates, isCommentSection logic, Back- action timing, PIN storage representation, AlarmManager API, and permission handling. Proposal descriptions must be updated if the source code differs.

5.8 Future AI Module Implementation Approach

Step 1. Provider evaluation: compare on-device and secured API options against structured-output support, extraction accuracy, latency, cost, privacy, availability, and Android integration effort. The provider is not selected at proposal stage.

Step 2. Context construction: implement a limited context builder that queries only the required task fields, user preferences, or aggregated statistics. Do not send the full database or accessibility-node tree.

Step 3. Structured response: require machine-readable fields for task drafts and schedule proposals. Reject malformed, incomplete, or unsupported actions before they reach the UI.

Step 4. Validation and confirmation: pass schedule proposals through the existing overlap algorithm; display all changes for user approval before invoking DatabaseHelper.

Step 5. Fallback and logging: allow manual entry when AI is disabled or unavailable. Store only operational diagnostics needed for debugging and avoid logging prompts containing personal information.

Step 6. Security: if a remote model is selected, route requests through an authenticated backend; never embed provider credentials in the Android APK.

CHAPTER 6: TESTING AND EVALUATION

6.1 Testing Strategy

Testing will combine unit, integration, system, UI, permission, service-lifecycle, database, and physical-device testing. Test data should include normal schedules, adjacent intervals, overlapping intervals, recurring tasks, empty days, completed days, and deliberately selected target/non-target application screens. Each test record should identify device model, Android version, target SDK, app build, permission state, and timestamps.

6.2 Testing Types

Unit testing: verify interval overlap, end-time calculation, status transitions, completion percentages, weekly aggregation, streak rules, preference parsing, and application-specific identifier matching. Integration testing: verify MainActivity-to-DatabaseHelper operations, PreferenceManager-to-service configuration, AppBlockerService-to-UsageStatsManager processing, overlay attachment/removal, ReelsBlockerService accessibility handling, and AlarmHelper-to-NotificationManager delivery. System and UI testing: cover first launch, permission grants, task creation, blocker activation, detection, analytics review, reminder receipt, navigation, theme changes, conflict messages, PIN entry, and blocking-screen usability. Permission testing: test granted, denied, revoked, and re-granted states; verify the main interface remains gated when either mandatory permission is absent. Background service testing: test service start/stop, notification visibility, process recreation, screen off/on, battery optimisation, and Android background-start restrictions. Detection testing: include target Reels/Shorts screens, ordinary feeds, comment sections, and cases where accessibility nodes are unavailable. Database testing: verify schema creation, CRUD operations, upgrades, invalid values, empty results, persistence after restart, and aggregate consistency. Performance testing: measure event-to-detection, event-to-overlay, event-to-Back latency, CPU, memory, wakeups, and battery drain.

6.3 Proposed Test Cases (TC-01 to TC-12)

Table 6.1. Proposed test cases TC-01 to TC-12.

ID	Scenario	Input / setup	Expected output	Status
TC-01	First launch without Usage Access	Usage Access denied	Permission guidance shown; main UI gated	Not Tested
TC-02	First launch without overlay access	Overlay permission denied	Permission guidance shown; main UI gated	Not Tested
TC-03	Grant required permissions	Both mandatory permissions granted	Main interface opens after re-check	Not Tested
TC-04	Create valid task	Study, 09:00, 60 min, None	Task saved in SQLite	Not Tested
TC-05	Adjacent tasks	09:00–10:00 and 10:00–10:30	No overlap reported	Not Tested
TC-06	Overlapping tasks	09:00–10:00 and 09:45–10:15	Conflict detected; second task not saved	Not Tested
TC-07	Task status before start	Current time before start	Status Pending	Not Tested
TC-08	Task status during interval	Current time inside interval	Status Ongoing	Not Tested
TC-09	Selected app blocking	Open selected blocked package	Overlay appears after package detection	Not Tested
TC-10	Unselected app access	Open unselected package	No blocking overlay	Not Tested
TC-11	YouTube Shorts detection	Open supported Shorts interface	Pattern detected and Back action attempted	Not Tested
TC-12	Normal YouTube feed	Open ordinary feed	No Shorts interception	Not Tested

6.4 Proposed Test Cases (TC-13 to TC-24)

Table 6.2. Proposed test cases TC-13 to TC-24.

ID	Scenario	Input / setup	Expected output	Status
TC-13	Facebook comment exception	Open comment section	Comment interaction remains accessible	Not Tested
TC-14	Instagram Reels detection	Open supported Reels interface	Pattern detected and Back action attempted	Not Tested
TC-15	TikTok feed detection	Open supported video feed	Detection rule evaluated and action attempted	Not Tested
TC-16	Accessibility node unavailable	Trigger event with null/unavailable source	Service handles safely without crash	Not Tested
TC-17	Daily percentage	3 completed out of 4 total	75% displayed	Not Tested
TC-18	Zero-task day	0 completed, 0 total	No divide-by-zero; neutral state displayed	Not Tested
TC-19	Streak calculation	Three consecutive qualifying daily records	Streak equals 3 under documented rule	Not Tested
TC-20	Task reminder	Schedule reminder with permission	Notification delivered under platform timing rules	Not Tested
TC-21	Exact alarm access denied	Exact alarm permission unavailable	Fallback or clear explanation	Not Tested
TC-22	PIN validation	Correct and incorrect four-digit values	Correct accepted; incorrect rejected	Not Tested
TC-23	Service lifecycle	Stop blocker or revoke overlay permission	Overlay removed safely; service state updated	Not Tested
TC-24	Android compatibility	Test suite on Android 10–15 devices	Version-specific behaviour documented	To Be Evaluated

6.5 Evaluation Metrics

Table 6.3. Evaluation metrics and measurement methods.

Metric	Definition / method
Detection precision	TP / (TP + FP): proportion of triggered detections that are correct.
Detection recall	TP / (TP + FN): proportion of target interfaces successfully detected.
False-positive rate	FP / (FP + TN), using a defined set of non-target screens.
Detection latency	Time from accessibility event receipt to detection decision.
Action latency	Time from event receipt to Back-action invocation; compare with ~500 ms target.
App-blocking latency	Time from relevant usage event becoming available to overlay display.
Battery impact	Difference in battery consumption between controlled enabled/disabled runs.
Task correctness	Percentage of task operations and status transitions matching expected results.
Usability	User feedback on permission clarity, task workflow, blocker usefulness, and interruption acceptability.

No empirical results are claimed in this proposal. All test cases are marked Not Tested or To Be Evaluated until evidence is collected from the implemented build. A zero-millisecond notification timeout must not be interpreted as zero detection latency.

CHAPTER 7: PROJECT PLANNING AND EXPECTED OUTCOMES

7.1 Development Timeline

The following 16-week plan is indicative and prioritises the core Android application. The AI assistant is a separately proposed extension and is not assumed to fit within the core 16-week implementation baseline. If approved, AI prototyping and evaluation should be scheduled after core stability or as a separately authorised development phase.

Table 7.1. Indicative 16-week development schedule.

Weeks	Phase	Main activities	Deliverable
1–2	Requirements and audit	Confirm source code, features, schema, target SDK, permissions	Verified requirements and baseline
3–4	Architecture and data	Finalise diagrams, SQLite schema, PreferenceManager responsibilities	Architecture and data design
5–6	Planner module	Task CRUD, recurrence, conflict checks, status management	Testable task planner
7–8	Analytics and reminders	Daily history, weekly summaries, streak, alarms, notifications	Analytics and reminders
9–10	Permission and app blocker	Permission gate, foreground service, usage monitoring, overlay	Application blocker
11–12	Reels detection	Accessibility rules, exceptions, Back action, warning	Reels detection module
13–14	Integration and testing	Cross-module tests, Android 10–15 checks, performance measurements	Test evidence and issue log
15–16	Documentation and defence	Final report, diagrams, references, demonstration preparation	Submission and presentation

7.2 Indicative Gantt Chart

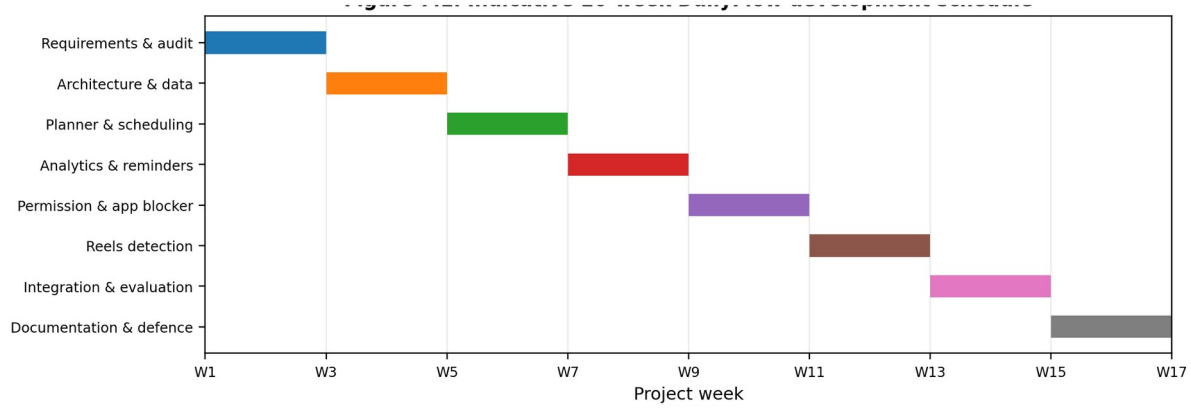


Figure 7.1. Indicative 16-week development schedule.

The chart uses sequential two-week work packages for planning. The approved project start date should replace any illustrative calendar dates. In practice, documentation, integration, and test design should run in parallel with implementation.

7.3 Expected Outcomes

- A working native Android application for category-based routine and task management.
- A local SQLite data layer for tasks and daily productivity aggregates.
- A preference layer for profile, theme, blocker configuration, package selection, and PIN-related settings.
- A UsageStatsManager-based selected-application monitoring path with overlay blocking.
- An AccessibilityService-based prototype for detecting specified short-video interfaces in supported applications.
- Daily and weekly productivity summaries, completion percentage, and streak calculation.
- A documented permission workflow, test matrix, compatibility findings, and performance evaluation.
- A documented design for a future AI assistant covering seven productivity use cases, controlled data access, validation, user confirmation, and an AI-specific test plan; this is a proposed design outcome, not a claim of implemented AI functionality.

7.4 Potential Challenges

- Third-party applications may rename or restructure UI resource identifiers and node hierarchies.
- Some content may not expose useful accessibility nodes or may expose incomplete information.
- UsageStatsManager event availability and polling introduce detection latency.
- Android foreground-service restrictions and vendor battery management may interrupt monitoring.
- Overlay permission revocation or window-type differences may prevent blocking UI display.
- Exact-alarm and notification permissions vary by Android version and target SDK.
- Frequent accessibility inspection and polling may increase battery use.
- PIN storage through SharedPreferences requires security hardening.
- A future AI service may introduce inaccurate suggestions, ambiguous task extraction, latency, service costs, data-transfer risks, or dependency on network/model availability.

7.5 Risk Analysis and Mitigation

Table 7.2. Project risk register and mitigation measures.

Risk	Likelihood / impact	Mitigation
Third-party UI changes	High / High	Keep rules isolated; regression-test after updates; document unsupported versions.
False-positive blocking	Medium / High	Use multiple indicators, comment exceptions, and non-target regression cases.
Missed short-video screens	High / Medium	Measure recall; record unavailable-node cases; state limitations.
OS terminates monitoring	Medium / High	Follow foreground-service rules; test vendor battery settings.
Permission denied/revoked	Medium / High	Re-check on resume; disable affected feature safely; explain access.
Battery drain	Medium / Medium	Tune polling/event processing; compare enabled/disabled consumption.
PIN exposure	Medium / High	Avoid plaintext storage; use salted derivation, rate limits, and no credential logging.
Schedule inconsistency	Low / Medium	Use transactions; validate recurrence dates and interval boundaries.

7.6 Future Enhancements

- Harden PIN storage using a salted password-based key derivation scheme and rate limiting.
- Add user-configurable focus schedules and temporary override workflows with clear feedback.
- Maintain application-specific short-video detection rules and regression tests as third-party interfaces change.
- Add richer local reports such as category-level completion trends and planned-versus-completed time.
- Develop the proposed AI Daily Routine Assistant and Natural-language Task Creation as the first optional AI increment.
- Extend the AI module with schedule optimisation and productivity explanations after structured-output and validation tests pass.
- Evaluate Adaptive Focus Coaching, task-aware Distraction Prevention prompts, and the Personal AI Chatbot with opt-in controls and limited data access.
- Conduct a larger usability study with informed consent and appropriate institutional approval; evaluate robustness across more devices, locales, app versions, and accessibility configurations.
- Consider optional backup/export with explicit user control and privacy safeguards.

CHAPTER 8: CONCLUSION

8.1 Project Summary

DailyFlow proposes an integrated Android productivity system that combines routine scheduling, selected- application blocking, and short-video interface detection. Its architecture separates user-interface coordination, local data management, configuration storage, background services, and Android system APIs. SQLite stores structured task and daily-history records, while SharedPreferences stores small independent settings. UsageStatsManager supports package-level monitoring, and AccessibilityService supports inspection of exposed UI hierarchy information for specified third-party interfaces. The proposal also defines a future AI-powered productivity assistant with seven capabilities: routine suggestions, natural-language task creation, schedule optimisation, productivity analysis, adaptive focus coaching, task-aware distraction prompts, and a conversational assistant. These capabilities remain proposed extensions and do not form part of the current core implementation.

8.2 Expected Contributions

The expected contribution is a practical software-engineering implementation and evaluation of selective distraction management integrated with routine planning. The project demonstrates how Android platform components can be combined while documenting permission dependencies, background restrictions, privacy considerations, interface fragility, and measurement requirements. It also specifies an extensible AI-assistant design that uses minimal context, structured output, deterministic validation, and user confirmation, providing a clear basis for future development without overstating current functionality.

8.3 Final Remarks

DailyFlow should be evaluated as a bounded Android prototype, not as a universal content detector or a clinical treatment for problematic technology use. Its effectiveness depends on correct permission handling, reliable service execution, accessible UI metadata, and continued maintenance of third-party-specific detection rules. The proposed AI extension should be treated as a future, optional capability until its model, data governance, implementation, and test results are approved and verified. The final submission should report only results measured from the actual build and should align every schema, method, diagram, and feature claim with the source code.

REFERENCES

[1] M. Lyngs, G. Lukoff, P. Slovák, and others, “Digital self-control interventions for distracting media multitasking: A systematic review,” Journal of Computer Assisted Learning, vol. 37, no. 5, pp. 1217–1231, 2021, doi: 10.1111/jcal.12581.

[2] J. A. Roberts and M. E. David, “Technology affordances, social media engagement, and social media addiction: An investigation of TikTok, Instagram Reels, and YouTube Shorts,” Cyberpsychology, Behavior, and Social Networking, vol. 28, no. 5, pp. 318–325, 2025, doi: 10.1089/cyber.2024.0338.

[3] S. Arouch, D. Edgcumbe, S. Pezaro, and K. Maravic Da Silva, “Short-form video, cognitive and mental health outcomes: A systematic review,” BMJ Public Health, vol. 4, no. 3, Art. no. e004086, 2026, doi: 10.1136/bmjph-2025-004086.

[4] Android Developers, “Restrictions on starting a foreground service from the background,” Android Developers documentation. [Online]. Available: https://developer.android.com/develop/background-work/services/fgs/restrictions-bg-start. Accessed: Nov. 7, 2026.

[5] Android Developers, “Changes to foreground services,” Android Developers documentation. [Online]. Available: https://developer.android.com/develop/background-work/services/fgs/changes. Accessed: Nov. 7, 2026.